



Vision por Computador Aplicada a Software

Modulo 4 - Vision por Computador aplicada a proyectos de programacion

Nombre del alumno/a	
Fecha	
Enlace GitHub del repositorio	

Instrucciones: Ejercicio practico conceptual. Disena soluciones CV reales, analiza codigo y razona tus decisiones tecnicas. Cada bloque combina teoria aplicada con practica guiada.

BLOQUE 1 - Analisis de Codigo CV: Detecta y Corrige (3.5 + 3.8)

Lee el siguiente fragmento de codigo Python. Identifica TODOS los problemas que encuentras:

```
# archivo: image_analyzer.py
import google.generativeai as genai
import PIL.Image

genai.configure(api_key='AIzaSyAbcl23xyz...')
model = genai.GenerativeModel('gemini-pro-vision')

def analizar_imagen(ruta):
    while True: # analiza continuamente
        imagen = PIL.Image.open(ruta)
        response = model.generate_content([
            'Describe esta imagen con todo detalle posible',
            imagen
        ])
        print(response.text)

# Procesar todas las imagenes de la carpeta
import os
imagenes = os.listdir('/uploads/') # puede ser 10.000 archivos
for img in imagenes:
    analizar_imagen('/uploads/' + img)
```

N	Problema detectado	Como lo corregiras?
1	La API key esta escrita directamente en el codigo (<code>genai.configure(api_key='...')</code>). Esto es un riesgo grave de seguridad, porque la clave puede filtrarse en GitHub, logs o capturas.	Moveria la clave a una variable de entorno o a un gestor seguro de secretos. Asi el codigo no expone credenciales y se puede desplegar de forma profesional.
2	Se usa una libreria y un modelo desactualizados o legacy (<code>google.generativeai</code> y <code>gemini-pro-vision</code>). Eso puede provocar incompatibilidades, peor mantenimiento o incluso que el ejemplo deje de funcionar.	Actualizaria al SDK oficial actual y a un modelo vigente de Gemini compatible con entrada multimodal. Tambien revisaria la documentacion oficial antes de elegir el nombre exacto del modelo.
3	La funcion <code>analizar_imagen()</code> contiene un <code>while True</code> , asi que nunca termina. Eso haria que el programa se quede analizando la misma imagen de forma infinita, disparando el coste y bloqueando el resto del procesamiento.	Eliminaría el bucle infinito y haria que la funcion procese cada imagen una sola vez. Si quisiera reintentos, los implementaria con limite y control de errores, no con un bucle eterno.
4	El prompt es demasiado vago y excesivo: "Describe esta imagen con todo detalle posible". No esta alineado con una tarea concreta y genera respuestas largas, caras y dificiles de procesar automaticamente.	Rediseñaria el prompt para que sea especifico y orientado al objetivo real. Por ejemplo: pedir OCR, clasificacion o salida JSON con campos concretos. Asi baja el coste y mejora la utilidad de la respuesta.
5	No hay validacion ni manejo de errores al abrir imagenes ni al llamar a la API. Si un archivo esta corrupto, no es imagen, la API falla o hay rate limit, la app puede romperse completa.	Añadiria <code>try/except</code> , validacion de formato, control de timeouts, reintentos con backoff y registro de errores. Asi el sistema sigue siendo robusto aunque falle una imagen o una llamada externa.
6	Procesa todos los archivos de <code>/uploads/</code> sin filtrar, sin limitar volumen y de forma	Filtraria solo extensiones validas, limitaria lotes, evitaria reprocesados y controlaria concurrencia/rate limit. Tambien podria

secuencial. Si hay miles de archivos, el sistema sera lento, caro y poco escalable.	reducir tamaño de imagen antes de enviar y guardar resultados para no repetir llamadas.
--	---

1.2 - Escribe las correcciones clave

Sin escribir el código completo, indica las partes/lineas criticas que cambiarías y por que:

Correccion	Respuesta
Correccion 1	Sustituiria la API key hardcodeada por una variable de entorno (GEMINI_API_KEY) o un sistema de secretos. La razon es proteger credenciales y evitar fugas de seguridad.
Correccion 2	Actualizaria la integracion a un SDK y modelo actuales de Gemini. La razon es evitar dependencias legacy y asegurar compatibilidad, soporte y mantenimiento a futuro.
Correccion 3	Quitaria el while True dentro de analizar_imagen(). La razon es que una sola imagen no debe analizarse infinitamente; eso dispara coste, tiempo de ejecucion y riesgo de bloqueo.
Correccion 4	Reescribiria el prompt para pedir una salida util y estructurada. Por ejemplo, si el objetivo es OCR o analisis automatico, pediria JSON con campos definidos en vez de una descripcion larguísima.
Correccion 5	Añadiria validacion de archivos antes de abrirlos. Comprobaria si existen, si son imagenes reales y si tienen un tamaño/formato permitido.
Correccion 6	Añadiria manejo de errores en apertura de imagen y en la llamada a la

Corrección 7	API. Usaría try/except , control de timeouts y reintentos limitados para que una sola falla no rompa todo el lote.
Corrección 8	Filtraría la carpeta /uploads/ para procesar solo imágenes válidas y no cualquier archivo. Esto evita errores por archivos no compatibles.
Corrección 9	Controlaría el procesamiento masivo con lotes, colas o límites de concurrencia. La razón es evitar sobrecargar la API, caer en rate limits y generar costes innecesarios.
Corrección 10	Reduciría el tamaño o resolución de las imágenes antes de enviarlas si no hace falta máxima calidad. Así se mejora velocidad y se reduce coste sin perder precisión en muchos casos.
	Guardaría resultados ya procesados para no volver a analizar la misma imagen varias veces. Esto reduce llamadas repetidas y mejora eficiencia.

El código no solo tiene errores de sintaxis de arquitectura, sino problemas de producción: seguridad, coste, escalabilidad, mantenibilidad y resiliencia frente a fallos.

BLOQUE 2 - Diseña el Flujo CV Completo (3.2 + 3.4)

Caso práctico: Una empresa de logística quiere automatizar la lectura de albaranes escaneados. El sistema debe: extraer el número de pedido, la fecha y los artículos, detectar si el documento es ilegible o está incompleto, y generar una alerta si falta algún campo obligatorio.

Completa las 5 fases del flujo CV para este caso concreto:

Fase	Nombre de la fase	Que hace en ESTE sistema?	Decision técnica clave
1	Recepción de imagen	El sistema recibe el albarán escaneado que sube el usuario o que llega desde un escáner. También registra metadatos básicos como nombre de archivo, tamaño y formato.	Aceptar solo formatos permitidos (PNG, JPG, PDF si aplica) y rechazar archivos corruptos o demasiado pesados antes de enviarlos a la IA.
2	Preprocesamiento	Se prepara la imagen para que el OCR funcione mejor. Esto puede incluir comprobar nitidez, orientación, contraste, recorte de bordes y reducción de ruido.	Validar si la imagen es legible antes de llamar al modelo. Si está borrosa, incompleta o mal escaneada, conviene parar ahí y pedir una nueva imagen para no gastar llamadas inútiles.
3	Construcción del prompt	Se redacta una instrucción precisa para que el modelo extraiga solo los datos necesarios del albarán. Se indican los campos esperados y el formato exacto de salida.	Pedir respuesta en JSON con claves fijas y reglas claras. Así la app puede validar automáticamente el resultado y detectar campos faltantes.
4	Llamada a la API	La app envía la imagen junto con el prompt al modelo	Elegir un modelo multimodal adecuado para OCR

		multimodal para que lea el documento y devuelva la información estructurada.	documental y controlar timeout, reintentos y coste por llamada.
5	Validación y acción	La app revisa si el JSON es válido, si los campos obligatorios existen y si el contenido tiene sentido. Luego guarda el resultado o genera una alerta si falta información.	No confiar ciegamente en la respuesta del modelo. Hay que validar formato, consistencia de fecha, presencia de número de pedido y lista de artículos antes de usarlo en producción.

2.2 - Diseña el Prompt Multimodal

Escribe el prompt completo para extraer datos del albarán. Recuerda: instrucción clara, formato JSON, campos y reglas de validación:

```
response = model.generate_content([
    "...", # Tu instrucción aquí
    Image.open('albaran.png')
])

# Salida JSON esperada:
{
    "numero_pedido": "...",
    "fecha": "...",
    "articulos": [],
    "campos_faltantes": []
}
```

Extrae la información de este albarán y devuelve SOLO un JSON válido, sin explicaciones ni texto adicional. Debes identificar los siguientes campos: "numero_pedido", "fecha", "articulos", "campos_faltantes", "documento_legible", "documento_incompleto".

Reglas:

- 1) Si un campo no aparece claramente, no lo inventes; devuélvelo como cadena vacía o añádelo en "campos_faltantes".
- 2) "articulos" debe ser un array de objetos con los campos "nombre", "cantidad" y "referencia" si existen.
- 3) "documento_legible" debe ser true o false según la calidad visual del documento.
- 4) "documento_incompleto" debe ser true si faltan campos obligatorios o si el albarán aparece cortado.
- 5) La fecha debe devolverse en formato DD/MM/AAAA si es posible.

6) Si hay dudas o baja confianza, reflejalo en "campos_faltantes". Formato exacto esperado:

```
{ "numero_pedido": "", "fecha": "", "articulos": [ { "nombre": "", "cantidad": "", "referencia": "" } ], "campos_faltantes": [], "documento_legible": true, "documento_incompleto": false }
```

BLOQUE 3 - Disena una Aplicacion CV Real (3.4 + 3.5 + 3.6)

Caso practico: Tu equipo quiere construir un sistema de moderacion de contenido para una red social educativa. Debe: (1) detectar imagenes inapropiadas antes de publicarlas, (2) extraer texto de imagenes para verificar el lenguaje, (3) describir el contenido visual para el historial de moderacion, (4) clasificar el tipo de imagen (foto personal, meme, documento, etc.). Recibira ~2000 imagenes/dia.

3.1 - Arquitectura: elige API y modelo para cada funcionalidad

Funcionalidad	API / Modelo elegido	Justificacion tecnica
Moderacion: detectar imagenes violentas o adultas	OpenAI Moderation API	Es una opcion pensada especificamente para filtrar contenido sensible con buena integracion y respuesta rapida. Para una red social educativa, la moderacion debe ser prioritaria y automatica antes de publicar.
OCR: extraer texto de la imagen	Gemini 2.5 Flash	Es una buena opcion multimodal para leer texto en imagenes sin entrenar nada y permite combinar OCR con instrucciones precisas sobre formato de salida.
Descripcion: generar resumen del contenido visual	GPT-4o mini	Es adecuado para describir imagenes en lenguaje natural

		con buen equilibrio entre calidad, velocidad y coste.
Clasificación: tipo de imagen (foto, meme, doc...)	Hugging Face ViT o clasificación zero-shot con GPT-4o mini	Si las categorías son simples, un modelo de clasificación dedicado o zero-shot es suficiente y mas barato que usar un modelo grande para todo.
Calidad: detectar si la imagen es ilegible o corrupta	Validación local + Gemini 2.5 Flash si hace falta confirmación	Primero conviene hacer comprobaciones locales de formato, peso, apertura y resolución. Si la imagen abre pero hay dudas de legibilidad, el modelo puede ayudar a detectar si esta borrosa, incompleta o inutilizable.

3.2 - Diseño modular (Principio de Responsabilidad Unica)

Divide la aplicación en módulos. Para cada uno indica su responsabilidad única y que llamada a la API realiza:

Nombre del módulo	Responsabilidad única (SRP)	Llamada a la API que realiza
Módulo de ingesta de imagen	Recibir la imagen, validar formato, tamaño y metadatos básicos.	No llama a la API; hace validación inicial local.
Módulo de moderación visual	Analizar si la imagen contiene violencia, desnudos o contenido sensible antes de publicarse.	Llama a la API de moderación para decidir si bloquear, revisar o permitir.

Modulo OCR	Extraer el texto que aparece dentro de la imagen.	Llama a Gemini con prompt orientado a OCR y salida estructurada.
Modulo de descripcion visual	Generar un resumen claro del contenido visual para historial o revision humana.	Llama a GPT-4o mini con prompt de descripcion breve y funcional.

3.3 - Optimizacion de costes

Con 2000 imagenes/dia, como controlarias el coste? Describe 4 estrategias concretas:

N	Estrategia de optimizacion	Como se implementa concretamente?
1	Reducir tamaño de imagen antes de enviarla	Redimensionar imagenes y comprimirlas antes de llamar a la API, siempre que no afecte a la tarea. Esto reduce coste, latencia y consumo innecesario.
2	Usar el modelo adecuado para cada tarea	No usar el modelo mas caro para todo. Por ejemplo, moderacion con una API especializada, OCR con Gemini Flash y clasificacion simple con un modelo mas ligero.
3	Filtrar localmente antes de llamar a la IA	Descartar archivos corruptos, formatos no validos o imagenes demasiado malas antes de pagar una llamada externa.
4	Evitar reprocesar la misma imagen	Guardar hash, ID o cache del resultado para no volver a analizar una imagen ya procesada. Asi se eliminan llamadas duplicadas.

BLOQUE 4 - Manejo de Errores en Vision por Computador (3.8)

Las APIs de CV pueden fallar por multiples razones. Completa la tabla indicando causa, impacto y solucion:

Error	Que lo causa?	Impacto en la app	Como lo manejarías?
Rate limit (429)	Demasiadas peticiones en poco tiempo o falta de control de concurrencia.	La app deja de procesar imágenes temporalmente, genera retrasos y puede perder tareas si no hay cola ni reintentos.	Aplicaría reintentos con backoff exponencial, cola de procesamiento y límite de concurrencia. También monitorizaría el volumen para repartir carga y evitar picos.
Imagen demasiado grande (400)	El archivo supera el tamaño o resolución admitidos por la API.	La petición falla antes de procesarse, aumenta la latencia y se desperdicia coste operacional.	Validaría tamaño antes del envío y haría compresión o redimensionado automático. Si sigue fuera de rango, devolvería un error claro al usuario.
Formato de imagen no soportado	El usuario sube un tipo de archivo no permitido o un archivo corrupto con extensión engañosa.	La llamada falla o el sistema intenta procesar un archivo inválido.	Validaría MIME type real, extensión y apertura local del archivo antes de llamar a la API. Si no es válido, lo rechazaría en la capa de ingesta.

OCR devuelve texto vacío	La imagen está borrosa, tiene poco contraste, no contiene texto útil o el modelo no pudo extraerlo correctamente.	La app podría guardar datos vacíos o tomar decisiones incorrectas si asume que "no había texto".	No asumiría ausencia de texto. Haría una segunda validación: revisar calidad de imagen, reintentar con mejor preprocesado y marcar el caso para revisión o nueva subida.
Score de confianza muy bajo	La detección o clasificación es ambigua, la imagen tiene mala calidad o el objeto aparece parcialmente oculto.	Existe alto riesgo de falsas decisiones, especialmente en moderación o validaciones críticas.	Definiría umbrales mínimos por caso de uso. Si el score queda por debajo, no automatizaría la decisión: lo enviaría a revisión manual o pediría una nueva imagen.
Respuesta JSON inválida	El modelo respondió con texto libre, JSON mal formado o campos fuera del esquema esperado.	La app puede romper el parser, guardar datos corruptos o fallar en cadenas automáticas posteriores.	Forzaría salida estructurada desde el prompt, validaría con esquema JSON y rechazaría cualquier respuesta no conforme. Si falla, reintento controlado o fallback seguro.
Timeout del servidor	La API tarda demasiado por carga alta, imagen pesada o red inestable.	La experiencia del usuario empeora, la tarea puede quedar a medias y se generan duplicados si el usuario reintentará sin control.	Usaría timeouts explícitos, reintentos limitados, idempotencia y estado de tarea asíncrona. Así evitamos bloqueos y duplicación de procesamiento.

4.2 - Verdadero o Falso: Buenas Prácticas CV

V / F	Afirmación	Justificación si es Falsa
F	Siempre es mejor enviar imágenes en máxima resolución para obtener mejores resultados de la API.	Es falsa porque una imagen demasiado grande aumenta coste y latencia y no siempre mejora la precisión. Lo correcto es enviar la calidad suficiente para la tarea, no la máxima posible.
V	Solicitar la respuesta en formato JSON facilita la validación y el procesamiento en la app.	
F	Un score de confianza de 0.4 es suficiente para tomar decisiones críticas de moderación.	Es falsa porque un 0.4 indica baja fiabilidad. En decisiones críticas conviene usar umbrales más altos o revisión humana para reducir falsos positivos y falsos negativos.
V	Hugging Face pipeline permite usar detección de objetos localmente sin llamar a una API externa.	
F	Si el OCR devuelve texto vacío, la aplicación debe asumir que la imagen no tiene texto.	Es falsa porque puede haber texto ilegible, mala calidad, reflejos o un fallo del OCR. Antes de asumirlo, hay que validar la imagen o reintentar.
F	Validar la salida de la IA antes de usarla es opcional si el proveedor es de confianza.	Es falsa porque incluso un buen proveedor puede devolver respuestas ambiguas, incompletas o

mal formadas. En producción, validar siempre es obligatorio.

En producción no se diseña para el caso ideal, sino para los fallos previsibles: límites de API, entradas malas, respuestas ambiguas y decisiones con incertidumbre.

BLOQUE 5 - Proyecto Integrador Final

Caso practico: Eres el/la desarrollador/a de una startup InsurTech que quiere lanzar un MVP en 2 semanas de un sistema de peritaje visual automatico. Los usuarios fotografian los danos en su vehiculo tras un accidente y el sistema debe: describir los danos detectados, estimar la gravedad (leve/moderada/grave), extraer la matricula del vehiculo y detectar si la foto es valida (no borrosa, no manipulada). Presupuesto limitado, ~500 peritajes/dia en fase inicial.

Decision tecnica	Tu respuesta justificada
Que proveedor y modelo eligiras para cada funcionalidad?	Usaria una arquitectura mixta para equilibrar coste, velocidad y calidad. Para describir los danos del vehiculo y estimar su gravedad, elegiria un modelo multimodal como GPT-4o mini o Gemini 2.5 Flash, porque pueden analizar imagen + instruccion y devolver una salida estructurada. Para extraer la matricula, usaria OCR con Gemini 2.5 Flash o una API especializada de documentos/vision si vemos que la matricula sale pequeña o con mala calidad. Para validar si la foto es apta, primero aplicaria validaciones locales de calidad y, solo si hay dudas, un modelo multimodal para decidir si la imagen esta borrosa, mal encuadrada o manipulada.
Que parametros clave configurarias (prompt, formato de salida)?	El prompt debe ser muy concreto y pedir una salida JSON obligatoria. No pediria texto libre. Solicitaria campos como descripcion_danos, gravedad, matricula, foto_valida, motivo_invalidez y confianza. Tambien incluiria reglas: no inventar matriculas,

	marcar gravedad solo entre leve, moderada o grave, y explicar por que una foto no es valida. Esto permite que la app valide facilmente y tome decisiones automáticas sin depender de texto ambiguo.
Como estructurarias los modulos de la aplicacion?	Separaria la aplicacion en modulos con responsabilidad unica. Un modulo de ingesta recibiria la foto y validaria formato y tamaño. Un modulo de calidad visual comprobaria nitidez, encuadre y si la imagen parece manipulada. Un modulo OCR leeria la matricula. Un modulo de analisis de daños generaria la descripcion y clasificacion de gravedad. Un modulo de validacion final comprobaria que el JSON sea correcto y que la respuesta tenga sentido. Y un modulo de orquestacion decidiria si aprobar, pedir nueva foto o enviar a revision manual. Esta separacion mejora mantenimiento, testeo y escalabilidad.
Que tecnica CV usarias para validar si la foto es valida?	Usaria una combinacion de validacion clasica + analisis multimodal. Primero, tecnicas locales sencillas: comprobar resolucion minima, apertura correcta del archivo, desenfoque, contraste y si la imagen esta demasiado oscura o cortada. Despues, si supera ese filtro, pediria al modelo una validacion funcional del tipo: "indica si la foto permite evaluar daños del vehiculo y leer la matricula". Esto es mejor que usar solo IA o solo vision clasica, porque reduce coste y mejora robustez.
Que errores anticipas y como los manejarias?	Anticiparia varios: fotos borrosas, matriculas no visibles, daños poco claros, respuestas JSON

	invalidas, timeouts y rate limits. También puede haber fotos duplicadas o usuarios que suban imágenes que no son del vehículo. Lo manejaría con validación previa, umbrales de confianza, reintentos limitados, schema validation del JSON y estados claros como foto invalida, requiere revisión o procesada. Nunca automatizaría una decisión crítica si la confianza es baja.
Como controlarías el coste con 500 peritajes/día?	Aplicaría varias medidas. Primero, redimensionar y comprimir imágenes antes de enviarlas. Segundo, usar el modelo más barato suficiente para cada tarea, evitando modelos grandes si no aportan valor. Tercero, no repetir análisis de la misma imagen: guardaría hash o cache de resultados. Cuarto, filtraría localmente fotos malas para no pagar llamadas inútiles. Y quinto, usaría flujo escalonado: solo si la validación local pasa, entonces llamamos al modelo multimodal. Esto reduce mucho el coste manteniendo calidad.